1. `isin()`

Use: Filter rows where a value exists in a list/set of values.

```
df[df["city"].isin(["Delhi", "Mumbai", "Pune"])]
```

NOT in the list

```
df[~df["city"].isin(["Delhi", "Mumbai"])]
```

Multiple values

```
df[df["department"].isin(["IT", "HR", "Finance"])]
```

With a Series

```
df[df["id"].isin(other_df["id"])]
```

Remember:

```
df["column"].isin([...])
```

returns a Boolean Series.

---

2. `between()`

Use: Filter values within a range.

```
df[df["age"].between(18, 30)]
```

By default, both boundaries are included:

```
18 <= age <= 30
```

Exclude boundaries

```
df[df["age"].between(18, 30, inclusive="neither")]
```

Only greater than 18, less than/equal to 30

```
df[df["age"].between(18, 30, inclusive="right")]
```

Numeric example

```
df[df["salary"].between(50000, 100000)]
```

Dates

```
df[
    df["date"].between("2025-01-01", "2025-12-31")
]
```

---

3. `query()`

Use: Filter DataFrames using a SQL-like syntax.

Instead of:

```
df[(df["age"] > 25) & (df["salary"] > 50000)]
```

you can write:

```
df.query("age > 25 and salary > 50000")
```

Multiple conditions

```
df.query("age > 25 and city == 'Delhi'")
df.query("age > 25 or salary > 100000")
```

`isin()` equivalent

```
df.query("city in ['Delhi', 'Mumbai', 'Pune']")
```

`not in`

```
df.query("city not in ['Delhi', 'Mumbai']")
```

Using Python variables

Prefix the variable with `@`:

```
min_salary = 50000

df.query("salary >= @min_salary")
```

Column names with spaces

```
df.query("`Annual Salary` > 50000")
```

Important

Inside `query()`:

```
and
or
not
```

instead of:

```
&
|
~
```

---

4. `at[]`

Use: Access or modify **one specific cell** using row label + column label.

```
df.at[5, "salary"]
```

Equivalent conceptually to:

```
df.loc[5, "salary"]
```

Modify a cell

```
df.at[5, "salary"] = 75000
```

Another example

```
df.at[10, "city"] = "Delhi"
```

Why use `at`?

For **single-value access/update**, it's optimized and convenient.

```
df.at[row_label, column_label]
```

---

5. `iat[]`

Use: Access or modify **one cell using integer positions**.

```
df.iat[2, 3]
```

Meaning:

```
row position = 2
column position = 3
```

Modify

```
df.iat[2, 3] = 100
```

Difference

Method	Uses
`at[]`	Labels
`iat[]`	Integer positions
`loc[]`	Labels, potentially multiple cells

Method	Uses
`iloc[]`	Integer positions, potentially multiple cells

Example:

```
df.at[101, "salary"]   # label
df.iat[2, 4]           # position
```

📥 Reading Data

6. `read_csv()`

Use: Read CSV files into a DataFrame.

```
import pandas as pd

df = pd.read_csv("employees.csv")
```

Common options

```
df = pd.read_csv(
    "employees.csv",
    sep=",",
    header=0,
    index_col=None
)
```

Select columns

```
df = pd.read_csv(
    "employees.csv",
    usecols=["name", "age", "salary"]
)
```

Parse dates

```
df = pd.read_csv(
    "employees.csv",
```

```
    parse_dates=["date"]
)
```

Missing values

```
df = pd.read_csv(
    "employees.csv",
    na_values=["NA", "N/A", "-", "missing"]
)
```

CSV without header

```
df = pd.read_csv(
    "data.csv",
    header=None
)
```

---

7. `read_excel()`

Use: Read Excel files.

```
df = pd.read_excel("employees.xlsx")
```

Specific sheet

```
df = pd.read_excel(
    "employees.xlsx",
    sheet_name="Employees"
)
```

Sheet by position

```
df = pd.read_excel(
    "employees.xlsx",
    sheet_name=0
)
```

Read multiple sheets

```
data = pd.read_excel(
    "employees.xlsx",
    sheet_name=None
)
```

Now:

```
data["Employees"]
data["Sales"]
```

returns individual DataFrames.

---

8. `read_json()`

Use: Read JSON data.

```
df = pd.read_json("data.json")
```

JSON from URL

```
df = pd.read_json(
    "https://example.com/data.json"
)
```

Different JSON structures

Sometimes:

```
df = pd.read_json(
    "data.json",
    orient="records"
)
```

Common orientations include:

```
records
columns
index
split
values
table
```

---

9. `read_sql()`

Use: Read SQL query results into Pandas.

```
df = pd.read_sql(
    "SELECT * FROM employees",
    connection
)
```

Query with conditions

```
df = pd.read_sql(
    """
    SELECT name, salary
    FROM employees
    WHERE salary > 50000
    """,
    connection
)
```

Read a table

```
df = pd.read_sql(
    "employees",
    connection
)
```

Depending on the database/connection setup, `read_sql_table()` may also be appropriate.

Typical workflow

```
import pandas as pd
from sqlalchemy import create_engine

engine = create_engine("...")

df = pd.read_sql(
    "SELECT * FROM employees",
    engine
)
```

🌐 10. Reading From URLs

Many Pandas readers can work directly with URLs.

CSV

```
url = "https://example.com/data.csv"

df = pd.read_csv(url)
```

JSON

```
url = "https://example.com/data.json"

df = pd.read_json(url)
```

Excel

```
url = "https://example.com/data.xlsx"

df = pd.read_excel(url)
```

Pro tip

You don't necessarily need to manually download the file first.

```
df = pd.read_csv(url)
```

📤 Writing Data

11. `to_csv()`

Use: DataFrame → CSV.

```
df.to_csv("output.csv")
```

Don't save the index

Usually useful:

```
df.to_csv(
    "output.csv",
    index=False
)
```

Specific columns

```
df.to_csv(
    "output.csv",
    columns=["name", "salary"],
    index=False
)
```

---

12. `to_excel()`

```
df.to_excel(
    "output.xlsx",
    index=False
)
```

Specific sheet

```
df.to_excel(
    "output.xlsx",
    sheet_name="Employees",
    index=False
)
```

---

13. `to_json()`

```
df.to_json("output.json")
```

Records format ⭐

Very common when working with APIs:

```
df.to_json(
    "output.json",
    orient="records"
)
```

Produces conceptually:

```
[
    {"name": "Amit", "age": 25},
    {"name": "Rahul", "age": 30}
]
```

Pretty JSON

```
df.to_json(
    "output.json",
    orient="records",
    indent=4
)
```

---

14. `to_sql()`

Use: DataFrame → SQL database table.

```
df.to_sql(
    "employees",
    con=engine,
    index=False
)
```

Replace existing table

```
df.to_sql(
    "employees",
    con=engine,
    if_exists="replace",
    index=False
)
```

Append data

```
df.to_sql(
    "employees",
    con=engine,
    if_exists="append",
    index=False
)
```

`if_exists`

```
"fail"     → error if table exists
"replace"  → replace table
"append"   → add rows
```

📁 15. Understanding File Paths

A **relative path**:

```
df = pd.read_csv("data/employees.csv")
```

Means:

```
current_folder/
    data/
        employees.csv
```

Parent directory

```
df = pd.read_csv("../data/employees.csv")
```

`..` means:

```
go one folder up
```

Absolute path — Windows

```
df = pd.read_csv(
    r"C:\Users\Rahul\Documents\data.csv"
)
```

or:

```
df = pd.read_csv(
    "C:/Users/Rahul/Documents/data.csv"
)
```

Using `pathlib` ⭐

Better for cross-platform code:

```
from pathlib import Path

path = Path("data") / "employees.csv"

df = pd.read_csv(path)
```

Check current working directory

```
import os

os.getcwd()
```

List files

```
os.listdir()
```

🔁 16. `duplicated()`

Use: Find duplicate rows.

```
df.duplicated()
```

Returns:

```
False
False
True
False
True
```

Show duplicate rows

```
df[df.duplicated()]
```

Check duplicates based on one column

```
df[df.duplicated("email")]
```

Consider multiple columns

```
df[df.duplicated(
    subset=["name", "email"]
)]
```

Keep last occurrence

By default:

```
df.duplicated()
```

keeps the **first** occurrence as non-duplicate.

```
df.duplicated(keep="last")
```

Mark all copies as duplicates

```
df.duplicated(keep=False)
```

This is extremely useful:

```
df[df.duplicated("email", keep=False)]
```

🗑️ 17. `drop_duplicates()`

Use: Remove duplicate rows.

```
df = df.drop_duplicates()
```

Based on one column

```
df = df.drop_duplicates(
    subset=["email"]
)
```

Keep last

```
df = df.drop_duplicates(
    subset=["email"],
    keep="last"
)
```

Remove all duplicated values

```
df = df.drop_duplicates(
    subset=["email"],
    keep=False
)
```

Modify original DataFrame

```
df.drop_duplicates(inplace=True)
```

🔄 18. `replace()`

Use: Replace values.

Single replacement

```
df["gender"] = df["gender"].replace(
    "M", "Male"
)
```

Multiple replacements

```
df["gender"] = df["gender"].replace({
    "M": "Male",
    "F": "Female"
})
```

Replace throughout DataFrame

```
df = df.replace({
    "Delhi": "New Delhi",
    "Bombay": "Mumbai"
})
```

Replace missing-like values

```
df = df.replace(
    ["NA", "N/A", "-", "unknown"],
    pd.NA
)
```

Regex replacement

```
df["phone"] = df["phone"].replace(
    r"\D",
    "",
    regex=True
)
```

🧹 19. Fixing Inconsistent Values

Real datasets often contain:

```
Delhi
delhi
DELHI
 Delhi
Delhi
New Delhi
```

A typical cleaning workflow:

```
df["city"] = (
    df["city"]
    .astype("string")
    .str.strip()
```

```
    .str.lower()
)
```

Now:

```
Delhi
DELHI
 delhi
```

become:

```
delhi
delhi
delhi
```

Then standardize:

```
df["city"] = df["city"].replace({
    "bombay": "mumbai",
    "calcutta": "kolkata"
})
```

Check unique values

```
df["city"].unique()
```

Count values

```
df["city"].value_counts()
```

⭐ Excellent cleaning pattern:

```
df["city"] = (
    df["city"]
    .astype("string")
    .str.strip()
    .str.lower()
    .replace({
        "bombay": "mumbai"
    })
)
```

🔢 20. Handling Incorrect Data Types

Check types:

```
df.dtypes
```

Detailed information:

```
df.info()
```

String → Integer

```
df["age"] = pd.to_numeric(
    df["age"]
)
```

If invalid values exist:

```
df["age"] = pd.to_numeric(
    df["age"],
    errors="coerce"
)
```

Invalid values become:

```
NaN
```

String → Float

```
df["salary"] = df["salary"].astype(float)
```

Safer:

```
df["salary"] = pd.to_numeric(
    df["salary"],
    errors="coerce"
)
```

String → Datetime ⭐

```
df["date"] = pd.to_datetime(
    df["date"]
```

```
)
```

Invalid dates:

```
df["date"] = pd.to_datetime(
    df["date"],
    errors="coerce"
)
```

Datetime → String

```
df["date"] = df["date"].dt.strftime(
    "%Y-%m-%d"
)
```

String → Category

Useful for repeated categorical values:

```
df["city"] = df["city"].astype("category")
```

Check one column

```
df["salary"].dtype
```

🏆 21. `nlargest()`

Use: Get rows with the largest values.

```
df.nlargest(5, "salary")
```

Means:

> Give me the top 5 employees by salary.

Top 10

```
df.nlargest(10, "sales")
```

Multiple columns

```
df.nlargest(
    5,
    ["salary", "age"]
)
```

Compared with sorting

This:

```
df.sort_values(
    "salary",
    ascending=False
).head(5)
```

is conceptually similar to:

```
df.nlargest(5, "salary")
```

For a straightforward top-N numeric query, `nlargest()` is cleaner.

🥇 22. `nsmallest()`

Opposite of `nlargest()`.

```
df.nsmallest(5, "salary")
```

Means:

> Give me the 5 employees with the lowest salary.

Cheapest products

```
df.nsmallest(
    10,
    "price"
)
```

Multiple columns

```
df.nsmallest(
    5,
    ["price", "rating"]
)
```

⚡ One-Page Memory Table

Function	Main purpose	Example
`isin()`	Match against multiple values	`df[df.city.isin(["Delhi","Pune"])]`
`between()`	Filter range	`df[df.age.between(18,30)]`
`query()`	SQL-like filtering	`df.query("age > 25")`
`at[]`	Single cell by label	`df.at[5,"salary"]`
`iat[]`	Single cell by position	`df.iat[5,2]`
`read_csv()`	Read CSV	`pd.read_csv("x.csv")`
`read_excel()`	Read Excel	`pd.read_excel("x.xlsx")`
`read_json()`	Read JSON	`pd.read_json("x.json")`
`read_sql()`	Read SQL	`pd.read_sql(query, conn)`
`to_csv()`	Save CSV	`df.to_csv("x.csv")`
`to_excel()`	Save Excel	`df.to_excel("x.xlsx")`
`to_json()`	Save JSON	`df.to_json("x.json")`
`to_sql()`	Save to SQL	`df.to_sql("table", engine)`
`duplicated()`	Detect duplicates	`df.duplicated()`
`drop_duplicates()`	Remove duplicates	`df.drop_duplicates()`
`replace()`	Replace values	`df.replace({"M":"Male"})`
`nlargest()`	Top N	`df.nlargest(5,"salary")`
`nsmallest()`	Bottom N	`df.nsmallest(5,"salary")`

🧠 The 10 patterns I'd memorize first

```python
# 1. Multiple values
df[df["city"].isin(["Delhi", "Mumbai"])]

# 2. Range
df[df["age"].between(18, 30)]

# 3. Complex filtering
df.query("age > 25 and salary > 50000")

# 4. Single cell by label
df.at[5, "salary"]

# 5. Single cell by position
df.iat[5, 2]

# 6. Find duplicates
df[df.duplicated("email", keep=False)]

# 7. Remove duplicates
df.drop_duplicates(subset=["email"])

# 8. Clean inconsistent strings
df["city"] = df["city"].str.strip().str.lower()

# 9. Top 5
df.nlargest(5, "salary")

# 10. Convert bad numeric data safely
df["salary"] = pd.to_numeric(
    df["salary"],
    errors="coerce"
)
```